

TourPlanner – Project Protocol

Semester Project - Software Engineering

Team: 07

Students: Iselin Sophie Magnussen, Matej Krsteski

Git repository: <https://github.com/iselinmag/SoftwareEngineering-TourPlanner.git>

Abstract

This protocol documents the final project architecture, selected technical decisions, failures, solutions, tests, diagrams and current implementation state. Tours and tour logs are readable by all users, while create/update/delete are restricted to the owning user; this shared-read/owner-write model was confirmed with the professor as an acceptable interpretation of the "no sharing" requirement.

Table of Contents

1. INTRODUCTION..... 3

2. REQUIREMENTS COVERAGE 3

3. ARCHITECTURE OVERVIEW 4

 3.1 BACKEND ARCHITECTURE 4

 3.2 FRONTEND ARCHITECTURE..... 4

4. UML USE CASE DIAGRAM 5

5. UML CLASS DIAGRAM..... 6

6. SEQUENCE DIAGRAM - FULL-TEXT SEARCH..... 7

7. UI FLOW AND WIREFRAMES 7

8. TECHNICAL DECISIONS..... 9

9. UNIQUE FEATURE: PER-TOUR IMAGE GALLERY.....10

10. DESIGN PATTERNS10

11. UNIT TESTS.....11

12. CONFIGURATION AND SECRETS.....11

13. CHALLENGES AND SOLUTIONS12

14. LIMITATIONS AND FUTURE IMPROVEMENTS12

15. TIME TRACKING12

16. CONCLUSION14

1. Introduction

TourPlanner is a two-tier web application for planning tours and documenting completed tours. Users register, log in, create tours between two locations, display route information on a Leaflet map, add tour logs, attach images to logs, search tour and log data, and import/export tour data as JSON.

The backend is Java 21 with Spring Boot, Spring Data JPA/Hibernate, PostgreSQL, Spring Security/JWT, Log4j2 and JUnit/Mockito tests. The frontend is Angular with services, standalone components and signal-based ViewModels following an MVVM-style architecture.

The application derives popularity and child-friendliness from tour logs. These values are calculated in the service layer instead of being persisted, so search combines a repository pass for stored fields with a service-layer pass for computed values.

2. Requirements Coverage

Requirement	Status	Project state
Angular + Spring Boot two-tier app	Done	Angular frontend in frontend/; Spring Boot backend in backend/.
MVVM frontend	Done	TourListViewmodel and TourLogViewmodel hold signal state and intent methods.
Layered backend	Done	Controller -> service -> repository. Controllers do not call repositories directly.
Design patterns + reusable component	Done	Repository, DI, DTO, MVVM, Facade. Reusable LogCardComponent.
PostgreSQL + JPA ORM	Done	User, Tour and TourLog entities; repositories use Spring Data JPA.
OpenRouteService + Leaflet	Done	RouteService calls ORS; MapFacadeService isolates Leaflet rendering.
Register/login	Done	AuthController, AuthService, JwtService and JwtAuthFilter.
Tour and log CRUD	Done	TourController/TourService and TourLogController/TourLogService.
Validation	Done	Bean Validation in TourDTO, TourLogDTO and AuthRequestDTO; Angular forms on frontend.
Computed attributes	Done	Popularity and child-friendliness calculated in TourService from logs.
Full-text search incl. computed values	Done	JPQL over stored fields/log comments, then service-layer filter over computed labels.
Import/export	Done	JSON export/import includes tours and their logs.
Images on filesystem	Done	FileStorageService stores files externally; TourLog.imagePath stores filename.
20+ tests + logging + config	Done	35 backend unit tests; Log4j2; application.properties gitignored with example file.
No sharing between users	Done	Tours and tour logs are readable by all users (list/search/get/export), while create/update/delete remain owner-aware via CurrentUser checks. This shared-read/owner-write model was confirmed with

		the professor as an acceptable interpretation of "no sharing between users," alongside the alternative of scoping reads to the logged-in user only.
--	--	---

3. Architecture Overview

3.1 Backend architecture

Layer	Responsibility	Examples
Controller	HTTP endpoints, @Valid input validation, response status codes.	TourController, TourLogController, AuthController
Service / business layer	Business rules, route lookup, computed attributes, import/export, file storage, auth and ownership checks.	TourService, TourLogService, AuthService, RouteService, FileStorageService, CurrentUser
Repository / DAL	Only path to PostgreSQL; Spring Data JPA queries and persistence.	TourRepository, TourLogRepository, UserRepository

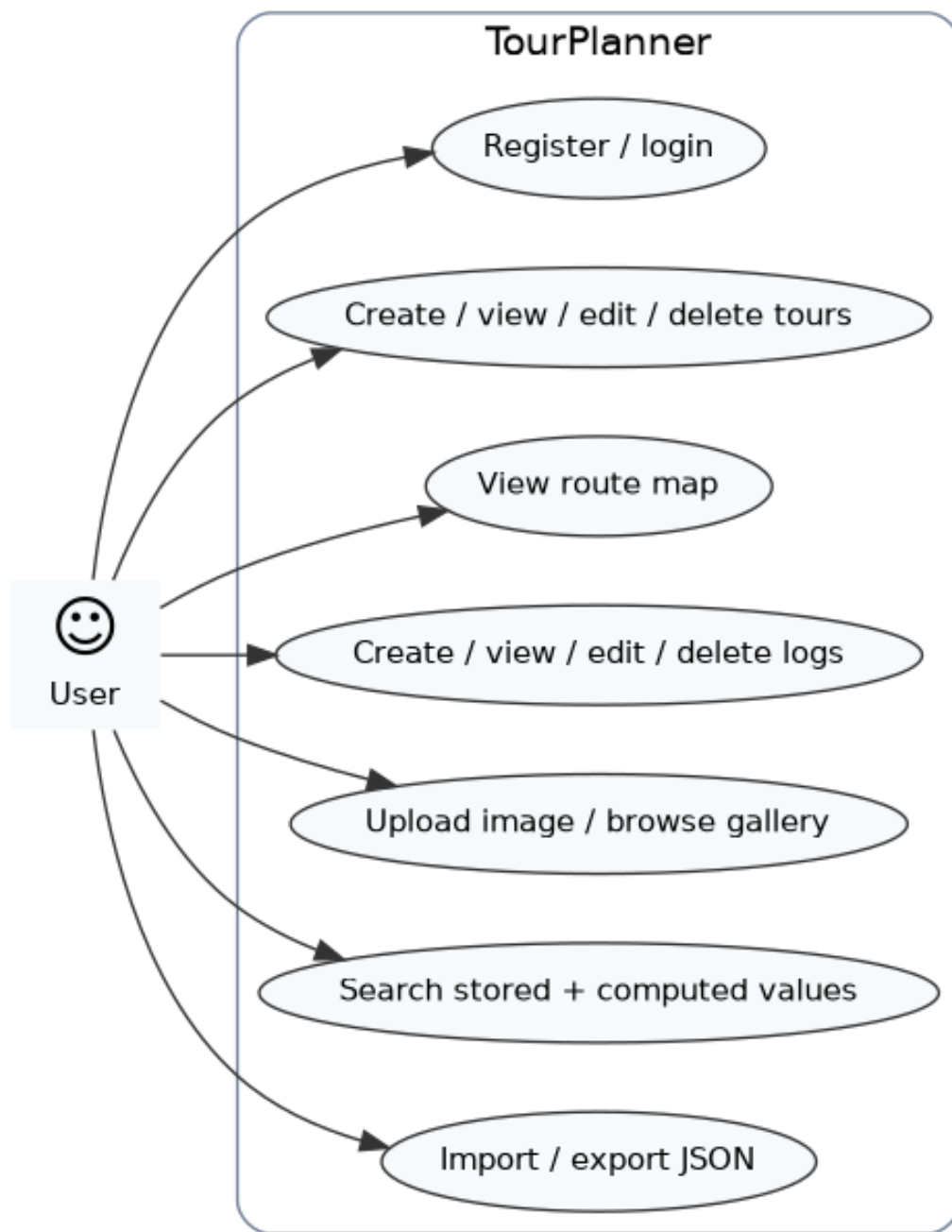
com.tourplanner

config/	SecurityConfig, WebConfig
controller/	REST endpoints
dto/	API DTOs and validation rules
entity/	JPA entities: User, Tour, TourLog
exception/	Domain exceptions + GlobalExceptionHandler
repository/	Spring Data JPA repositories
security/	JWT filter and token service
service/	Business logic, route lookup, file storage, current user

3.2 Frontend architecture

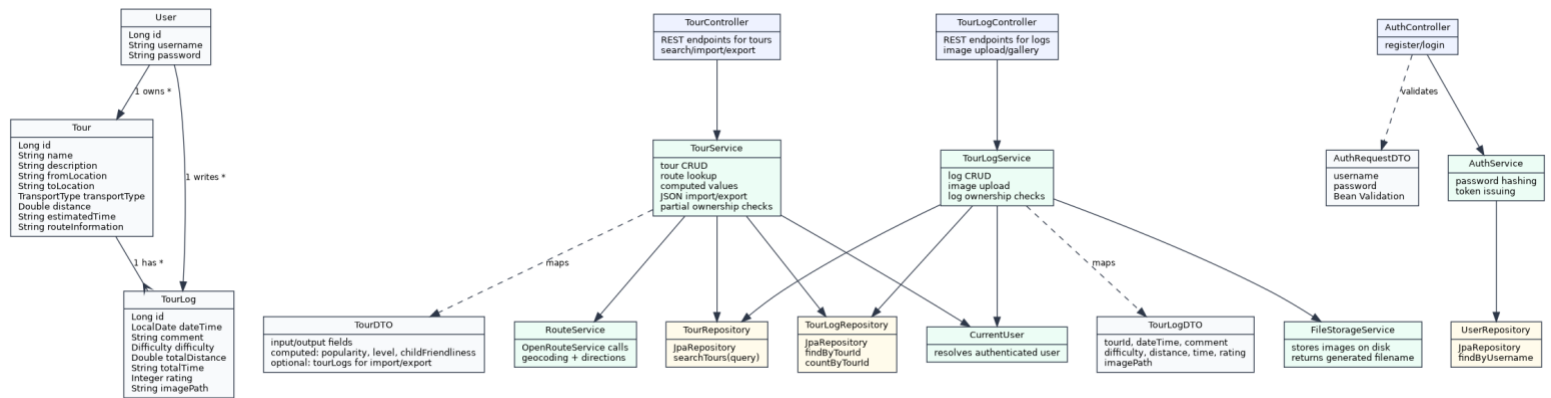
Components and templates are the View. TourListViewModel and TourLogViewModel are the ViewModel layer. Angular services and TypeScript models form the Model/integration layer. Leaflet-specific code is kept in MapFacadeService so TourMapComponent only reacts to selected-tour state and asks the facade to draw or clear routes.

4. UML Use Case Diagram

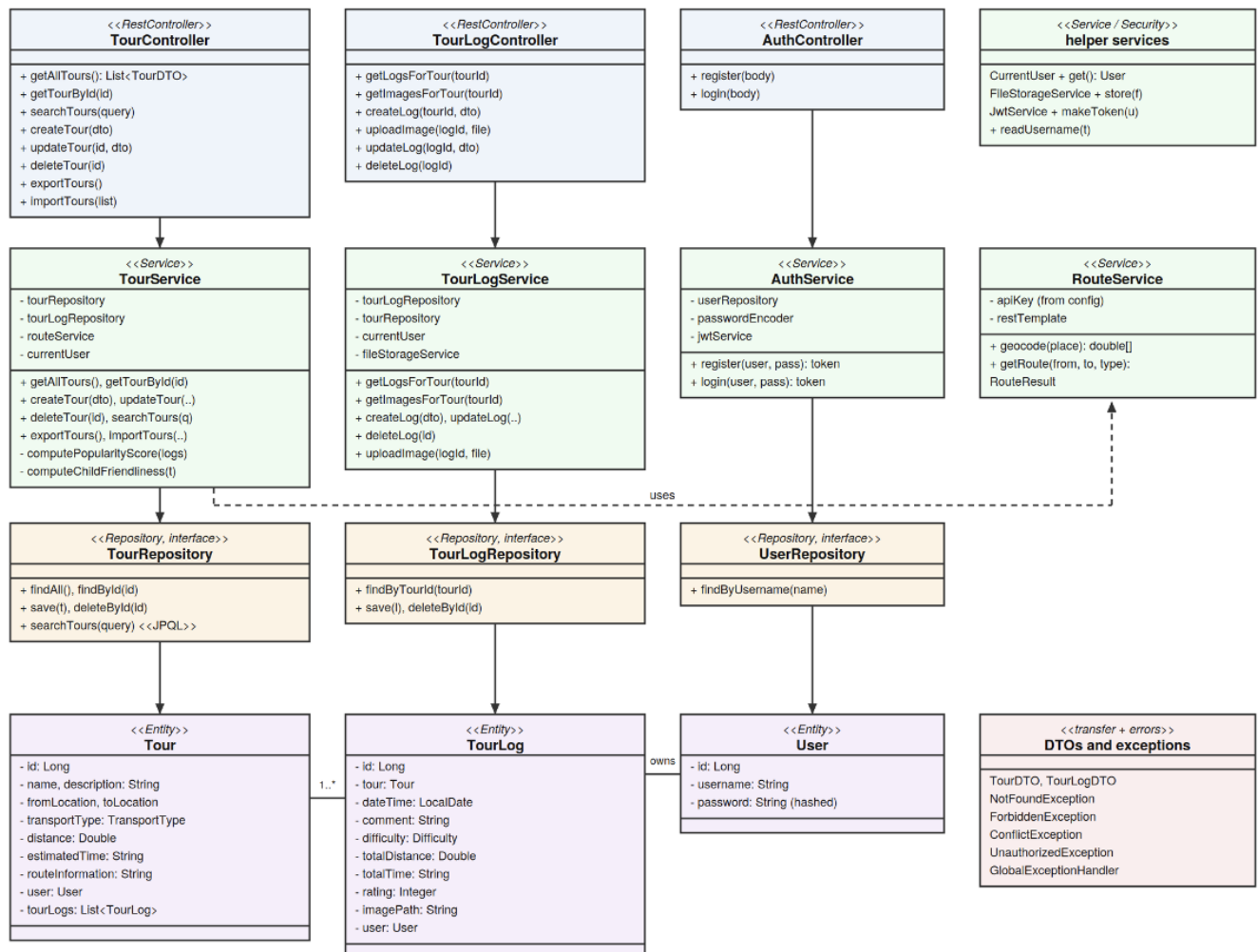


The registered user can authenticate, manage tours, view route maps, manage tour logs, upload images, search and import/export JSON data.

5. UML Class Diagram

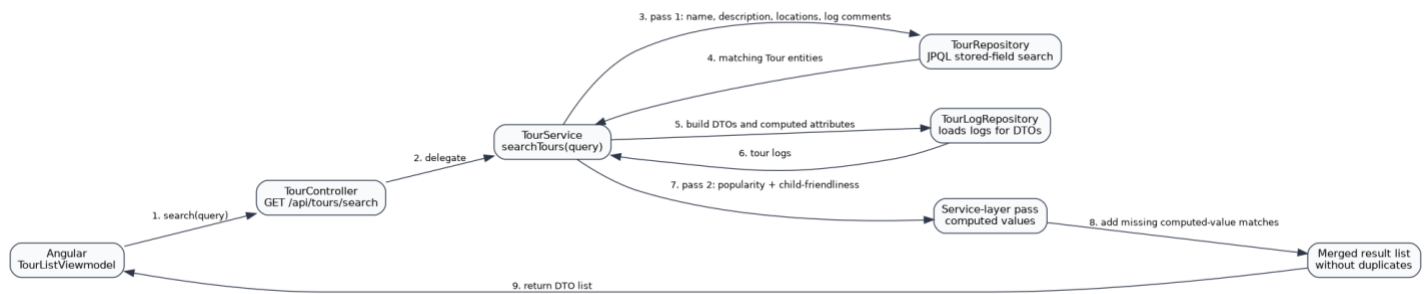


Backend class diagram (core classes)



The core data model is User -> Tour -> TourLog, with TourLog also storing the user who wrote the log. Services map entities to DTOs and keep computed values in the business layer.

6. Sequence Diagram - Full-Text Search



Search first uses a JPQL repository query over stored tour fields and tour-log comments. Because popularity and child-friendliness are computed at request time, TourService then builds DTOs and checks those computed labels in memory, merging new matches without duplicates.

7. UI Flow and Wireframes

Unauthenticated users use login/register screens. Authenticated users use the main screen with a tour list, search/import/export controls, map, selected-tour details, logs and image gallery.

Wireframe 1 - Login and Register Flow

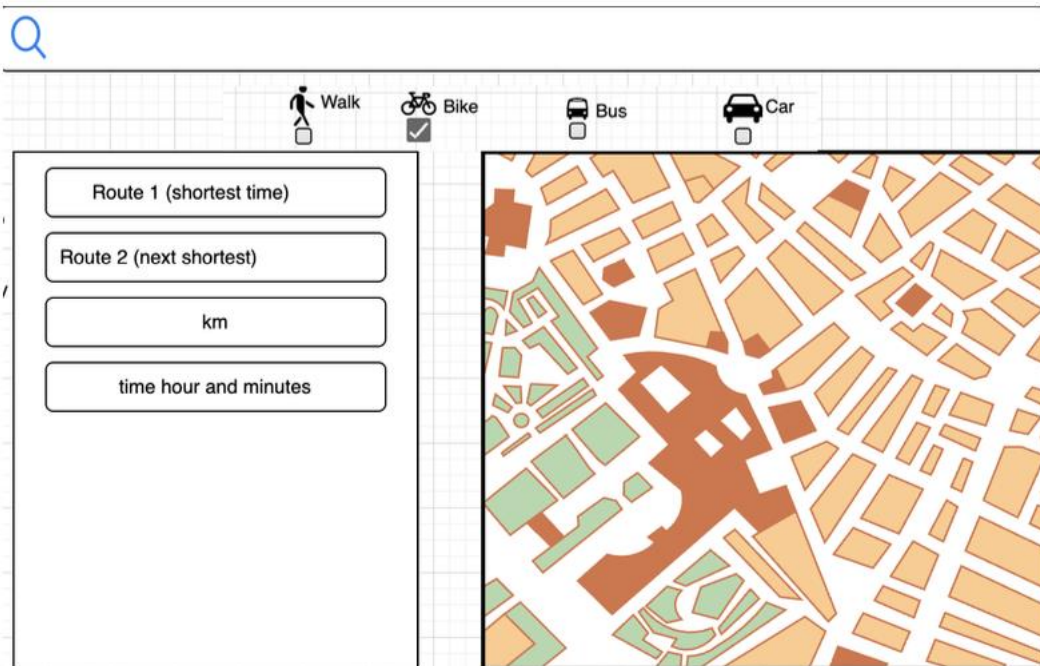
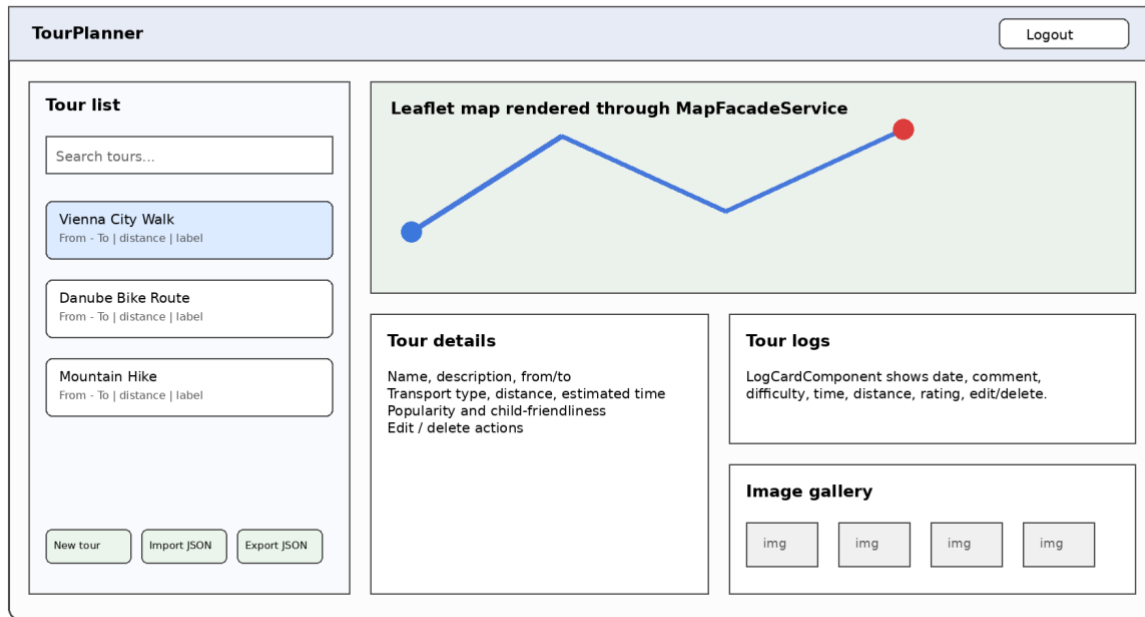
Login

[Create an account](#)

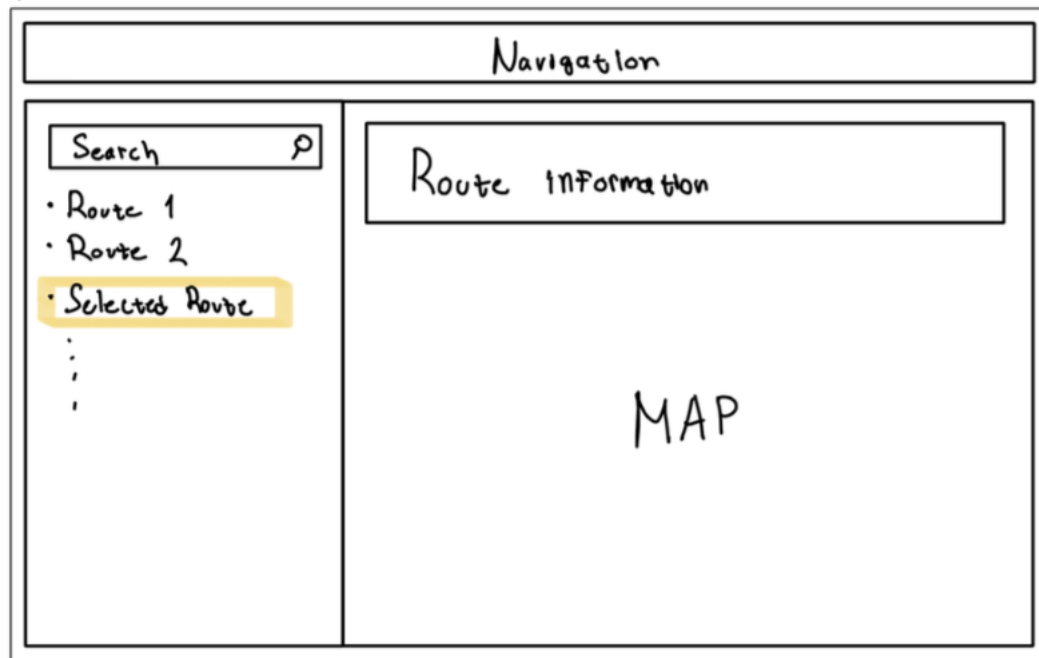
Register

[Already have an account? Login](#)

Wireframe 2 - Main Tour Planner Screen



Desktop View



Mobile view



8. Technical Decisions

- Thin controllers and service-owned business logic.

- Spring Data JPA repositories as the only database access layer.
- Bean Validation on DTOs instead of trusting frontend-only validation.
- JWT bearer authentication with stateless Spring Security filter chain.
- Externalized configuration for database, JWT secret, ORS key and upload directory.
- Signal-based ViewModels and computed state in the Angular frontend.
- MapFacadeService around Leaflet to isolate the external map library.

Decision	Reason	Trade-off
Manual DTO mapping	Computed fields require service logic and log lookups.	More boilerplate than MapStruct.
ddl-auto=update	Fast local setup while schema changed often.	Production should use Flyway and validate.
RestTemplate in RouteService	Small blocking MVC app with few ORS calls.	A larger app should use a dedicated typed client with explicit timeouts.
jjwt for JWT	Equivalent JWT library choice; structure still follows bearer-token architecture.	Different from the lecture library example.
One TourDTO for input/output	Simple for a small project.	Separate create/update/public DTOs would be stricter.
Long IDs	Readable in URLs and debugging.	Requires ownership checks to prevent unsafe access.

9. Unique Feature: Per-Tour Image Gallery

The unique feature is a per-tour image gallery. It does not only storing an image file externally, but connecting uploaded log photos into a selected-tour gallery so the user can visually browse memories for each tour. Users attach images to tour logs. FileStorageService saves uploaded files on the filesystem using generated filenames, and only the filename is stored in TourLog.imagePath. The frontend then displays images for the selected tour through TourImagesComponent.

This feature adds practical user value to a tour diary and demonstrates the required split between relational tour/log data in PostgreSQL and external image files on the filesystem.

10. Design Patterns

Pattern	Where used	Purpose
Repository	TourRepository, TourLogRepository, UserRepository	Separate persistence from business logic and enable mocked unit tests.
Dependency Injection	Constructor injection in controllers/services/security	Loose coupling and replaceable collaborators.
DTO	TourDTO, TourLogDTO, AuthRequestDTO	Stable API shape separate from JPA entities.

MVVM	Angular components + ViewModels + services/models	Separate view rendering from state coordination.
Facade	MapFacadeService	Hide Leaflet-specific setup, markers, polylines and cleanup.

11. Unit Tests

The backend has 35 JUnit/Mockito unit tests: 23 in TourServiceTest and 12 in TourLogServiceTest. The frontend also contains AppComponent and TourListViewmodel specs. The backend tests are the most important because they cover the service layer, where the project-specific logic lives.

Area	Count	Why critical
TourServiceTest	23	Tour CRUD, route fallback, computed values, search, import/export and ownership checks.
TourLogServiceTest	12	Log CRUD, image upload/gallery and log ownership checks.
Frontend specs	15 cases	App shell and ViewModel state/intent behaviour.

```
[INFO] Tests run: 12, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.000 s -- in com.tourplanner.service.TourLogServiceTest
[INFO] Running com.tourplanner.service.TourServiceTest
21:25:19.147 [main] ERROR com.tourplanner.service.TourService - Route service threw an exception, saving tour with user values: API down
[INFO] Tests run: 23, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.058 s -- in com.tourplanner.service.TourServiceTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 35, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 2.038 s
[INFO] Finished at: 2026-07-05T21:25:19+02:00
[INFO]
```

```
DEV v4.0.18 /Users/mk/Angular Projects/SWEN2-Tour-planner-FINAL/Untitled/SoftwareEngineering-TourPlanner/frontend
✓ tourplanner src/app/tour-list/tour-list-viewmodel.spec.ts (12 tests) 18ms
✓ tourplanner src/app/app.spec.ts (3 tests) 25ms

Test Files 2 passed (2)
Tests 15 passed (15)
Start at 21:26:15
Duration 791ms (transform 63ms, setup 283ms, import 66ms, tests 42ms, environment 866ms)
```

The tests use mocked repositories and mocked dependencies. They do not require a running PostgreSQL database, OpenRouteService account or filesystem state, which makes them fast and deterministic.

12. Configuration and Secrets

The real application.properties file is gitignored. The repository contains application.properties.example with placeholders for PostgreSQL credentials, OpenRouteService API key, JWT secret and upload directory. This keeps secrets outside source control and allows each developer to configure the app locally.

```
backend/src/main/resources/application.properties.example
backend/src/main/resources/application.properties    # local only, gitignored
```

```
tourplanner.upload.dir=...
openrouteservice.api.key=...
jwt.secret=...
spring.datasource.url=...
```

13. Challenges and Solutions

Challenge	Solution
Unclear auth errors	Added domain exceptions and GlobalExceptionHandler.
OpenRouteService coordinate order	Converted longitude/latitude geometry to Leaflet latitude/longitude rendering.
Export missed logs	Export/import now carries nested tour logs.
Computed values not searchable in JPQL	Added service-layer computed-value search pass.
Child-friendliness used too little data	Extended calculation to difficulty, distance and parsed time.
Wrong route-service stubs in tests	Strict Mockito stubbing exposed and corrected wrong arguments.
Map component coupled to Leaflet	Moved Leaflet logic to MapFacadeService.
Different local database setups	Used config example, ddl-auto=update and DB-independent unit tests.
TourLog table mapping risk	Tour maps to tours; TourLog must map to tour_logs.

14. Limitations and future improvements

The application fulfills all required features. Beyond the requirements, we identified a few areas we would extend if the project continued. We list them here to document where the current scope ends.

- **One image per tour log.** A log currently carries a single photo. The storage service and the gallery are built in a way that would allow a list of images per log, so extending this would mainly mean changing the image path field into a collection.

15. Time Tracking

Approximate time tracking

Activity	Matej (h)	Iselin (h)	Total (h)
Project setup and planning	1	4	5
Backend: entities, repositories, controllers	8	4	12
Backend: services and business logic	10	1	11
OpenRouteService and Leaflet integration	8	1	9
Authentication and ownership	7	1	8

Frontend: screens and ViewModels	8	8	16
Unit tests	1	7	8
Import/export, search, computed attributes	0	6	6
Unique feature (image gallery)	5	0	5
Debugging and polishing	6	4	10
Documentation and protocol	1	1	2
Sum	55	37	92

16. Conclusion

TourPlanner implements the main technical stack required by the semester project: Angular frontend, Spring Boot backend, JPA/Hibernate with PostgreSQL, OpenRouteService integration, Leaflet rendering, Log4j2 logging, externalized configuration, validation, authentication, JSON import/export and more than 20 unit tests.

Architecturally, the backend follows controller-service-repository layering and the frontend uses an MVVM-style signal architecture. The most important design choices are manual service-layer DTO mapping for computed values, a Facade around Leaflet, and mocked service tests for deterministic unit testing.